

ControlPlane.ai — An Adaptive Policy-Discovery Layer for Enterprise AI Governance

A design-rationale paper documenting the ideation journey, architecture, and prototype plan Prepared for the Accenture Innovation Challenge 2026

Abstract

Enterprise deployments of AI carry three recurring risks: the AI can be confidently wrong (**performance**), quietly expensive (**cost**), or in violation of organizational rules (**responsibility**) — usually discovered only after the fact. The obvious response, a real-time "AI checker" that inspects every response and blocks, edits, or escalates it, is already a crowded market (runtime guardrails, agent-behavior firewalls, PII/toxicity filters, model routers, audit trails). This paper documents how that starting point was systematically attacked, narrowed, and replaced with a different thesis: **the harder and less-occupied problem is not enforcing policy, but discovering which policy an organization should actually have**. We present ControlPlane.ai, an adaptive execution-policy layer that observes AI decisions, human interventions, and business outcomes; mines that evidence for statistically defensible patterns; drafts a human-readable policy proposal; and — only after explicit human approval — compiles and deploys the change as deterministic, versioned, enforceable policy. We document the ideas incorporated and discarded along the way, the competitive landscape that forced each narrowing, the resulting architecture (with an explicit AI/deterministic component boundary), a prototype plan scoped to prove the discovery loop rather than build enterprise infrastructure, and the lessons learned in the process.

1. Introduction

1.1 The problem as given

The challenge brief frames the task as building a technology layer — "ControlPlane Checker" — that continuously observes AI responses across performance, cost, and responsibility, sitting above any model, and asks three design questions:

- How should each risk category be detected?
- Should a flagged response be blocked, edited, or escalated?
- How can this happen without adding so much latency that AI becomes unusable?

1.2 Where this paper ends up

The central finding of the ideation process was that the brief, read literally, invites a large field of entrants toward the same crowded territory: PII detection, toxicity filtering, token caps, model routing, runtime guardrails, and observability dashboards. All of these are real, necessary, and already commercially mature. None of them, alone, is a defensible point of differentiation for a new entrant.

The reframing that survived adversarial scrutiny was:

Don't just build a system that checks whether AI obeyed policy. Build a system that learns which policy actually works for a given organization.

This produces the working definition used throughout the rest of this paper:

ControlPlane is an adaptive execution-policy layer for enterprise AI. It observes AI decisions, human interventions, and business outcomes; discovers recurring patterns in that evidence; proposes changes to the organization's executable AI policies; lets an authorized human approve, reject, or edit the proposal; then deterministically enforces the approved policy.

The loop is: **Observe → Discover → Propose → Approve → Enforce → Observe again.**

The single governing engineering principle is:

AI does discovery and drafting. Deterministic systems do enforcement. No generative model is ever the final authority over a consequential action.

2. Initial Vision and Approach

The first conceptual model treated ControlPlane as a **supervisor sitting between an AI and its user**: read the prompt, observe the response, score it on performance/cost/responsibility, and decide to allow, edit, block, or escalate — in milliseconds.

The guiding analogy was a **live-TV broadcast delay**: a short buffer that lets a producer intervene without halting the whole broadcast.

text

User → AI streams response → ControlPlane observes stream
→ potential redact/edit/intervene → User

This was useful for reasoning about latency and intervention granularity, and several of its structural ideas (a fast, low-latency path; a graded intervention scale) survived into the final architecture. But it was explicitly demoted as the *headline differentiator*, because synchronous inspection-and-intervention is already a well-represented product category — see Section 4.

3. Incorporated and Discarded Ideas

The project moved through eight named concepts before converging. Each is recorded here with its status and the specific reason it was kept, demoted, or rejected, because the reasoning is as valuable as the conclusion.

Idea	Description	Verdict	Reason
A — Live TV / streaming intervention	Inspect and redact AI output mid-stream (e.g. redact an SSN as it streams)	Demoted to supporting capability	PII redaction and runtime content filtering are already mature commodity capabilities.
B — Traffic-light intervention	RED/YELLOW/GREEN mapping to block/warn/allow	Demoted to UX concept	Risk-graded allow/block/modify/escalate is standard guardrail behavior industry-wide.
C — Micro-escalation	Live Slack/Teams approval request with countdown, instead of a ticket queue	Kept as possible supporting feature	Improves UX of human-in-the-loop but is not itself a defensible innovation.
D — Liability Black Box	Every intervention produces a forensic record (prompt, response, policy, evidence, decision, timestamps, approvals) for legal/insurance defensibility	Kept as necessary infrastructure	Audit trails and decision logs are already baseline in observability/evaluation tooling — required, not differentiating.
E — Behavioral Contracts	Explicit business rules such as "never quote a price without a database lookup" or "never issue a refund above threshold without approval"	Kept as conceptual ancestor of the final concept	First serious move away from generic safety toward business-specific rules — but a static contract is still policy-as-code, and invites the question "who wrote the contract?" This question became the doorway to the final concept.
F — Consensus Gate	High-risk agent actions require a second independent model/checker to co-sign before executing	Kept as a possible future extension	Visually compelling and relevant to agentic AI, but does not address the deeper problem of <i>evolving</i> organizational policy.
G — Cost as an attack surface	Treat recursive tool loops, retries, and wasteful model selection as a "denial-of-wallet" security problem	Kept as an evidence signal, not core	Reframes cost usefully but is not, alone, the central innovation.

Idea	Description	Verdict	Reason
H — Message recall	Deliver a response quickly, then let an asynchronous "deep lane" recall or edit it later	Rejected outright	A message may already have been read, and more fundamentally: you cannot recall a side effect. If an agent has already transferred money, deleted data, sent an email, or deployed code, editing the chat message afterward changes nothing. This surfaced a logical contradiction in any recall-based architecture.

3.1 The adversarial attack

At this point the team deliberately asked an AI collaborator to attack the concept as a hostile technical judge would. The resulting objections, and their consequences, were:

1. **The taxonomy is crowded** — allow/modify/verify/escalate/block already exists across guardrail products.
2. **Fast lane / deep lane is not novel** — synchronous moderation plus asynchronous evaluation is a common pattern.
3. **A threshold demo is not AI** — if the demo reduces to `if refund > ₹10,000: require_human()`, a judge is right to ask why this needed AI at all.
4. **Audit logs are not a moat** — logging is baseline infrastructure.
5. **Recall doesn't solve consequences** — post-delivery correction cannot reverse a completed action (formalizing the rejection of Idea H above).
6. **The hardest question is policy authorship** — who creates and maintains the behavioral rules in the first place? This question, unresolved by every idea up to this point, became the doorway to the breakthrough.

3.2 The breakthrough

The reframing insight was that **organizations already generate policy information through their real-world decisions**. If an AI repeatedly escalates certain transactions and humans repeatedly approve them, those approvals are evidence — not to be imitated blindly, but combined with context and downstream outcomes and mined for structure:

text

```
1,000 decisions → human overrides + transaction context + outcome data
→ pattern discovery
→ "Premium customer + airline fault + refund ≤ ₹15k is consistently
    approved and produces successful outcomes."
→ proposed policy
```

The organization never wrote this rule down. The operational data revealed it. This is the load-bearing idea of the entire project.

4. Related Work and Competitive Landscape

This section is explicitly maintained as a **living research section** — the AI governance market moves quickly enough that any "nobody does X" claim must be re-verified before final submission.

4.1 Runtime guardrails

Fiddler positions "Guardrails" as a runtime policy-enforcement layer for agentic AI, evaluating prompts and responses for hallucination, jailbreaks, PII, and unsafe content, with sub-80ms enforcement claimed in its environment (fiddler.ai/guardrails). **AWS Bedrock AgentCore** documentation shows deterministic policies consuming guardrail confidence scores and returning allow/deny decisions (docs.aws.amazon.com/bedrock-agentcore). The lesson drawn: combining a probabilistic detector with deterministic enforcement is already validated industry practice — this confirms the architecture is sound, but also means the *architecture alone* cannot be the pitch.

4.2 AI security / agent behavior

Lakera / Check Point AI Security document prompt defense, data-leakage prevention, agent behavior defense, tool allow/deny lists, off-task action detection, and runtime screening of tool calls and responses (docs.lakera.ai). This demonstrates that "govern the agent's *actions*, not just its text" is already an active product category, meaning consequence-control alone is not a sufficient differentiator either.

4.3 AI control planes

Fiddler's 2026 material describes an "AI agent control plane" combining identity, observability, security, and runtime enforcement (fiddler.ai/blog/ai-agent-control-plane-beyond-proxy). The term "control plane" itself is becoming a crowded phrase in this market — the differentiation claim must avoid asserting novelty of the *term* and instead assert novelty of the *learning loop* layered on top of it.

4.4 Adjacent research on learning from human overrides

A closely related direction was found in a vendor blog (Arize AI, "self-improving agent on a context graph of human disagreement," May 2026), which mines human override data into a context graph and proposes updates to system-prompt instructions and policy documents. This is the nearest published analogue to ControlPlane's core loop, with one meaningful difference: it proposes edits to *prompts and policy documents*, not to a structured, versioned, machine-enforced execution policy with an explicit accept/reject governance gate. An academic protocol (CHAP, arXiv) formalizes "override as learning data" with tagged, queryable correction records, but stops at structured logging rather than generating an executable policy artifact. Neither constitutes a shipped product doing the full loop described in Section 6.

4.5 Market implication

Current competitors are increasingly capable across runtime guardrails, policy enforcement, agent action monitoring, tool permissions, security screening, observability, and risk scoring. These must be treated as **foundational capabilities that any credible entrant needs, not as headline innovation**. The narrower whitespace being pursued is:

Historical organizational behavior + outcomes → discovered policy → human-approved executable policy evolution.

4.6 Differentiation statement

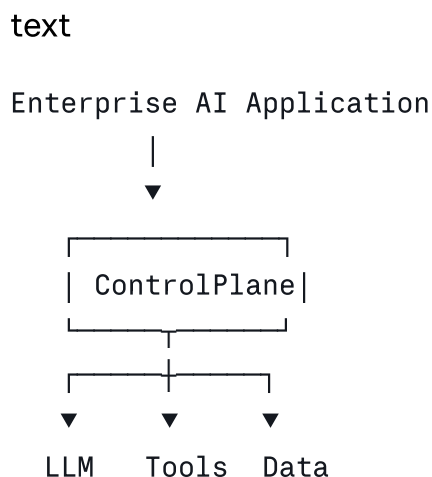
Existing guardrails enforce policies people define. Existing policy engines execute policies people write. ControlPlane's differentiator is discovering candidate policies from how an organization actually operates, validating those patterns against outcomes, and asking humans to approve the resulting policy evolution.

This is deliberately narrower and more falsifiable than a generic claim like "we have better AI safety."

5. ControlPlane.ai: System Overview

5.1 What it is and is not

ControlPlane is not a chatbot, a model provider, a generic observability dashboard, or a generic content filter. It is an **adaptive policy/control layer** sitting at the execution boundary of an enterprise AI application, ideally model-agnostic:



5.2 Who it is for

The target is **enterprises deploying AI agents into consequential business workflows** — insurance claims, procurement, financial operations, customer refunds, coding/deployment agents, healthcare administration — where an AI decision has a measurable downstream consequence. The product has no meaningful value proposition against low-stakes generative tasks ("write me a poem"); it requires actions like "process this claim," "approve this refund," or "deploy this change."

5.3 Domain selection methodology

Seven candidate demonstration domains were scored against six criteria: whether decisions are genuinely consequential, whether human overrides occur naturally, whether outcomes are measurable, whether multiple contextual factors matter, whether all three challenge dimensions (performance/cost/responsibility) are present, and whether a policy is genuinely *discoverable* rather than trivially hard-codeable.

Domain	Consequential	Overrides	Outcomes	Context	3 dimensions	Discoverability	Total
Insurance claims	5	5	5	5	5	5	30
IT / code agents	5	4	5	4	5	4	27
Healthcare administration	5	4	2	5	4	4	24
Financial operations	4	4	3	4	4	3	22
Customer refunds	3	5	3	3	4	3	21
Procurement	3	4	2	4	4	3	20
Customer support	2	2	3	2	3	2	16

Insurance claims scored highest on every axis, not just one: overrides (adjuster review) are institutionally normal, outcomes are rich and fast to reference (fraud confirmation, disputes, reopened claims), context is genuinely multivariate, and the domain is regulated enough to give the "responsibility" dimension real teeth. IT/code agents scored a close second and is retained as the natural "this generalizes to..." example in the pitch, since it demonstrates the architecture is not domain-specific.

Insurance is the proving ground for the prototype, not the entire addressable product.

6. Architecture

6.1 The central design decision: separate enforcement from learning

The single most important architectural decision in the project is that **runtime enforcement must be fully deterministic**. No generative model sits in the final approval path for a consequential action.

text

RUNTIME (fast path)

Request/action

↓

Current versioned policy

↓

Deterministic evaluation

↓

ALLOW / MODIFY / VERIFY /
ESCALATE / BLOCK

LEARNING (slow path)

Production traces

↓

Human interventions

↓

Business outcomes

↓

ML / pattern discovery

↓

Candidate policy

↓

LLM-generated explanation/diff

↓

Human review

↓

Regression testing

↓

Policy version → back to runtime

6.2 Component classification

Component	Technology class	Rationale
RBAC / auth	Deterministic	Binary authorization fact
Budget / threshold check	Deterministic	Numeric comparison
Schema validation	Deterministic	Structural correctness has one right answer
Current-policy evaluation	Deterministic	This is exactly what a policy engine should do
Fuzzy risk scoring	ML / anomaly model	Requires pattern recognition across correlated features nobody would hand-code
Evidence grounding	Retrieval + bounded LLM comparison	The LLM's role is narrow: does claim X match retrieved fact Y — not "is this okay"
Pattern discovery	ML / statistics / interpretable rule mining	Discovers conditional structure nobody explicitly programmed
Policy explanation / diff	LLM (generative)	Legitimate use — advisory prose a human reviews before anything changes
Approved-policy compilation	Deterministic codegen + regression tests	The semantic decision was already made by a human; turning it into executable syntax is mechanical and testable
Outcome attribution	Data joins + statistics/ML	Ground truth should come from business data, not from asking a model if a decision "seems fine"

6.3 Where AI is actually load-bearing

Four places, and only four:

1. **Pattern discovery** — finding multivariate relationships nobody explicitly programmed (e.g. $\text{premium tier} + \text{airline-caused incident} + \text{verified documents} + \text{refund} \leq ₹15k \rightarrow \text{high successful-approval probability}$)).
2. **Risk/anomaly detection** — flagging unusual combinations of factors that don't trip any known rule.
3. **Evidence comparison** — a bounded model comparing a claim against retrieved evidence and returning match/mismatch/uncertain, never the final governance decision.
4. **Policy proposal drafting** — turning discovered statistical structure into human-readable rationale, a policy diff, and suggested conditions, always subject to human

review.

6.4 Latency strategy

The brief explicitly asks how to avoid AI-defeating latency. The answer is architectural, not a performance optimization:

We don't make every request slower to make the system smarter. We make the system smarter between requests.

The fast path performs only deterministic evaluation against the *current* versioned policy. All learning, discovery, and proposal generation happens asynchronously against historical data and never blocks a live request.

6.5 Intervention taxonomy

The final principle is that the intervention itself is **policy-defined, not universally hard-coded**:

- **ALLOW** — low-risk, policy-compliant operation.
- **MODIFY** — safe transformation such as redaction, schema correction, or removing a non-critical unsafe element.
- **VERIFY** — require an additional deterministic or evidence-grounded check.
- **ESCALATE** — human decision required.
- **BLOCK** — hard stop for unacceptable risk.

The system should always select the least disruptive intervention compatible with the assessed risk.

6.6 Learning from humans without treating humans as ground truth

A serious objection is: *what if the humans were wrong?* The response is to treat human decision + context + outcome as **evidence**, not as truth. Candidate outcome signals include later fraud flags, chargebacks, complaints, successful completions, corrections, cancellations, financial loss, and downstream approval/rejection. The system should prefer patterns that correlate with *successful outcomes*, not merely with human agreement — making this **outcome-aware policy discovery** rather than "copy what humans did."

6.7 Policy proposal lifecycle

text

1. Discover → 2. Measure → 3. Validate → 4. Explain → 5. Human review
→ 6. Regression test → 7. Version → 8. Deploy → 9. Monitor

Worked example:

text

CURRENT POLICY

Claims < ₹50,000 → AI processing

Claims ≥ ₹50,000 → Human review

DISCOVERED PATTERN

Premium customer + verified documentation + airline-caused
cancellation + claim ≤ ₹75,000

→ Approval rate 98.7%, successful-outcome rate 97.9%, low intervention

PROPOSAL

Premium AND airline_fault AND verified_documents AND claim ≤ ₹75,000

→ AUTO-PROCESS

Human: ACCEPT / REJECT / EDIT → Policy v1 becomes Policy v2

Regression testing must confirm the new rule does not introduce unacceptable historical failures before it is versioned and deployed.

6.8 Full system diagram

text

CONTROLPLANE.AI

ENTERPRISE AI
/ AGENT SYSTEM



CURRENT POLICY
Deterministic runtime



ALLOW / MODIFY / VERIFY
/ ESCALATE / BLOCK



AI OPERATES



REAL-WORLD EVIDENCE
decisions · interventions ·
cost · performance ·
responsibility · outcomes



POLICY DISCOVERY ENGINE
ML + statistics + patterns



POLICY PROPOSAL
LLM explanation + diff



HUMAN AUTHORITY
REJECT | APPROVE



REGRESSION TEST → POLICY VERSION → RUNTIME

7. Prototype Philosophy and Proposed Tech Stack

7.1 Philosophy

The prototype does not need to build the enterprise product. It needs to prove exactly one thing:

A system discovers a useful policy that nobody explicitly programmed, explains it, gets human approval, and enforces it on a new, unseen case.

Everything else — connectors, IAM, multi-model routing, dashboards — is supporting infrastructure that can be simulated or omitted.

7.2 Minimal prototype architecture

text

Historical decisions + outcomes

↓

Policy Discovery Engine (ML / rule mining / statistical tests)

↓

Policy Proposal Generator (LLM explanation + policy diff)

↓

Human Review (ACCEPT / EDIT / REJECT)

↓

Regression Testing

↓

Versioned Policy

↓

Deterministic Runtime Engine

↓

New AI request

7.3 Proposed stack (provisional)

- **Frontend:** React dashboard — policy proposal view, policy diff, evidence visualization, approve/reject controls.
- **Backend:** Node.js/Express or Python/FastAPI.
- **Data:** PostgreSQL (or SQLite for the demo); CSV for initial experiments.
- **ML:** scikit-learn — decision trees / interpretable rule learners, anomaly detection, statistical significance testing. Interpretability is a deliberate choice over black-box models, since explainability is itself part of the pitch.
- **LLM:** any capable model (e.g. Claude), used strictly for explanation, natural-language proposal drafting, and evidence summarization — never as final authority.
- **Policy format:** start with JSON, e.g.:

json

```

{
  "conditions": [
    {"field": "customer_tier", "operator": "==", "value": "premium"},
    {"field": "incident_type", "operator": "==", "value": "airline_fault"},
    {"field": "claim_amount", "operator": "<=", "value": 75000}
  ],
  "action": "AUTO_PROCESS",
  "requires_human": false
}

```

with a later path to compiling this into a formal policy language (e.g. Cedar-style).

7.4 Demo sequence

1. Show the existing static policy (e.g. claims $\geq$ ₹50,000 $\rightarrow$ human review).
2. Replay thousands of historical decisions, interventions, and outcomes.
3. Run discovery and surface the multivariate pattern found.
4. Show the supporting evidence: sample counts, approval and outcome rates, confidence, held-out validation.
5. Present the policy diff.
6. Human (the judge) clicks Approve.
7. Show regression-test impact against historical data.
8. Feed a new, unseen case through the system and show the newly approved policy applied deterministically — the proof moment.

7.5 What would make the demo fake, and how to avoid it

The two failure modes to guard against are (a) scripting the discovery ("if premium: print('AI discovered premium')") and (b) hard-coding the exact expected pattern into a synthetic dataset and presenting it as if ML found it. The credible version uses a generated or realistic dataset with multiple variables, noise, and overlapping patterns, with held-out validation demonstrating measurable predictive lift over a naive single-threshold baseline (e.g. 82% precision for a flat threshold versus 94% for the discovered multivariate rule) — and the exact numbers must come from the actual experiment run, not be invented for the pitch.

8. Anticipated Objections

Objection	Response
"Isn't this just an if-statement?"	The deterministic engine is an if-statement; the innovation is discovering <i>which</i> if-statement should exist, from evidence nobody hand-coded.
"Why use AI at all?"	AI is confined to pattern discovery, anomaly detection, evidence comparison, and proposal drafting — see Section 6.3. Remove the AI and the discovery loop stops working; the enforcement layer keeps working (by design).
"Why trust the LLM?"	The LLM never has enforcement authority. Its output is advisory prose reviewed by a human before any policy changes; worst case is a badly worded draft that gets rejected.
"What if the humans are wrong?"	The system learns from human decisions <i>combined with outcomes</i> , not from human decisions as ground truth — see Section 6.6.
"Why isn't this just policy-as-code?"	Policy-as-code starts from a human-written rule and compiles it. ControlPlane discovers the candidate rule from behavioral and outcome evidence first, and only then compiles the human-approved result into policy-as-code.
"Why not just use an existing guardrail product?"	Existing guardrails enforce policies someone already wrote. None of the reviewed competitors mine override/outcome history into a versioned, human-approved, executable policy proposal — see Section 4.4–4.5.
"Why not let the AI automatically change the policy?"	This was deliberately rejected — see the Human Authority gate in Section 6, and Lesson 5 below. Unilateral AI policy authority reintroduces exactly the "what if it learns bad behavior" risk the project is trying to avoid.

9. Lessons Learned

1. **We became more innovative by removing features**, not by adding them. Every crowded-market rejection (Section 3) sharpened the surviving idea.
2. **A good architecture can still be a commoditized product**. "Fast lane + deep lane + risk score + block/edit/escalate" sounded sophisticated but, alone, was not enough — see Section 4.
3. **Always ask what happens if a judge removes the AI**. If the demo still works as five lines of `if` statements, the AI was never load-bearing.
4. **Don't use generative AI where deterministic systems are better**. This improved both technical credibility and actual safety.

5. **Human feedback is not automatically ground truth.** Outcome attribution matters — see Section 6.6.
 6. **The hardest enterprise problem may be policy evolution, not policy enforcement.** Writing the first rule is easy; keeping rules aligned with reality as conditions change is hard.
 7. **Latency is an architectural problem, not something to hand-wave.** Move expensive discovery and learning off the critical path entirely rather than trying to make it fast enough to run inline.
 8. **Don't compete on commodity capabilities.** Guardrails, PII filtering, quotas, routing, observability, and runtime blocking are foundations to build on, not the pitch.
 9. **A strong innovation can be a new loop, not a new model.** The proposed contribution is the feedback loop — operation → evidence → discovery → policy evolution → operation — not a novel ML technique.
-

10. Preserving Progress: Process Notes for Future Sessions

This project deliberately separated its record-keeping into three tiers, a pattern worth continuing:

- **Frozen** (do not change without a strong, explicitly stated reason): the core product thesis, the Observe→Discover→Propose→Approve→Enforce loop, the AI-vs-deterministic architectural boundary, the human-authority requirement, the three challenge dimensions as evidence sources, and insurance claims as the demonstration domain.
- **Still open** (expected and safe to iterate on): the exact insurance workflow, dataset design, specific ML algorithm, policy representation format, frontend/backend implementation, evaluation metrics, product name/tagline, prototype scope, and final pitch materials.
- **Do not reopen without strong reason** (already attacked and rejected — reopening without new evidence wastes time already spent): the generic AI firewall framing, message recall, PII filtering or token caps as *differentiation* (rather than supporting features), generic model routing, generic audit trails, and a broad "AI governance dashboard" framing.

The single sentence to keep at the top of any future working session:

■ **We are not replacing the (if). We are discovering which (if) should exist.**

And the project north star:

■ Observe how AI operates. Discover what actually works. Propose how policy should evolve. Let humans decide. Enforce deterministically. Learn again.

11. Current Status and Next Phases

Settled: vision, product thesis, differentiation claim, architecture (runtime/learning split), domain selection. **Not yet designed:** the concrete prototype.

The next phases, in order:

1. **Choose the exact insurance workflow** (claims triage, claim approval, fraud investigation, or reimbursement assessment).
 2. **Build the synthetic/realistic dataset**, including claim context, AI decision, human decision, model used, latency, cost, and downstream outcome.
 3. **Prove pattern discovery** — compare a simple threshold baseline against a discovered multivariate policy on precision, recall, approval prediction, outcome quality, human workload, and cost.
 4. **Generate the policy proposal** — LLM produces rationale, human-readable diff, and evidence summary.
 5. **Build the human approval UI** — accept/reject/edit.
 6. **Compile and regression-test** the approved policy into deterministic executable form.
 7. **Demonstrate runtime** — feed unseen cases through the newly deployed policy.
-

12. Conclusion

The project began with the question *"how do we build a better AI checker?"* and, through repeated adversarial narrowing against a genuinely crowded competitive landscape, arrived at a different and more defensible question: *"how do we build a system that learns what an organization has actually learned about how its AI should behave?"* The concept evolution ran:

text

AI checker → AI guardrail → Behavioral firewall → Behavioral contracts
→ Policy enforcement → Human feedback → Policy mining
→ Outcome-aware policy discovery → Adaptive AI policy layer

The final scope is intentionally narrower than the original brainstorm. That narrowing is treated as a feature: a system that mines behavioral and outcome evidence into a human-approved, deterministically enforced policy is a claim precise enough to be demonstrated, defended against a hostile technical judge, and distinguished from the current generation of guardrail, gateway, and control-plane products — while still directly answering all three dimensions (performance, cost, responsibility) named in the original challenge.

Appendix A — Research Caveat

The competitive-landscape section of this paper reflects the sources reviewed during this ideation process as of the current date. Some earlier competitive observations originated from other AI assistants' research and were not independently verified at the time they were first recorded. Before using any absolute claim (e.g. "nobody does X") in a final submission, a dedicated, dated competitive scan should be re-run and the claim phrased conservatively, since the AI governance and agent-security market is moving quickly.